01

System Architecture

How Miesign is built and hosted: the browser app, the API, the business layers, the data model, and the exact role Gemini plays — and does not play — in turning one house photo into a redesign and a priced estimate.

1 Overview and design principles
2 Deployment architecture
3 Backend layers
4 Data model
5 API surface (19 endpoints)
6 AI integration and guardrails
7 Request lifecycle: rendering a design
8 Frontend architecture
9 Concurrency, cost and security
10 Technology stack

PRODUCT
Miesign — AI exterior renovation planner and INR cost estimator

AUTHOR
Mihir Kapildev Tiwari

LIVE
miesign.pages.dev · github.com/MihirKT/Miesign

1 · OVERVIEW

Overview and design principles

Miesign is a web application for Indian homeowners. A user uploads one photo of the front of a house, reviews the building parts the system detects, chooses materials and paint shades for each part, sees their own house redesigned, and receives material quantities and an itemised cost estimate for their city, downloadable as a PDF.

Principle 1 — the AI never does arithmetic

Gemini only *classifies and describes*: it finds boxes around parts, turns words into a structured design, rewrites an idea, finds published rates, and edits the photo. Every area, quantity and rupee is computed by pure, unit-tested Python in `estimation.py`, so every number is reproducible.

Principle 2 — every AI answer is constrained and re-checked

Each Gemini call has a JSON response schema (materials limited to the catalog, parts limited to those present), and the server validates the result again before saving. Rates from search are accepted only within 0.3×–3× of a known base rate.

Principle 3 — the user can correct everything

Detected parts can be dragged, retyped, added or deleted; the scale can be set from one real measurement; every material and labour rate can be overridden; designs can be edited and re-rendered.

Principle 4 — free to run

One always-free Google Cloud e2-micro server, Cloudflare Pages for the public site, SQLite on the server disk, and Gemini on Vertex AI billed per call against free credits. No paid hosting, no API keys stored anywhere.

Deployment architecture

The public address is **miesign.pages.dev**. Cloudflare Pages serves the built React app and runs a small Pages Function that forwards every `/api/*` request to the Django server, so the app and its API share one origin (no CORS). The server is a single Google Cloud e2-micro VM; Caddy terminates HTTPS and passes requests to gunicorn. Django calls Gemini on Vertex AI using the VM's own service account, so no credentials exist on disk or in the repository.

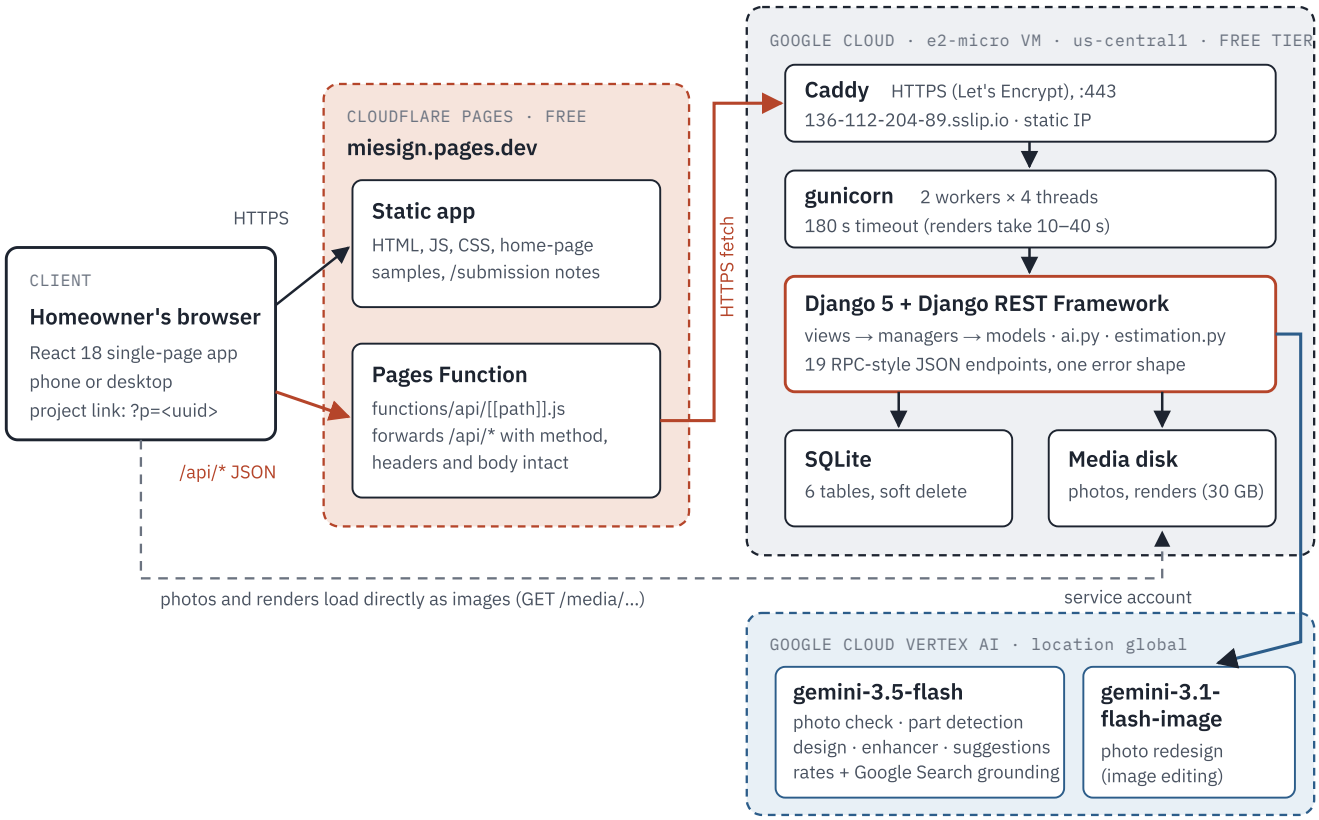


Fig 1.1 Deployment. Solid orange = API path through the Pages proxy; blue = server-to-Gemini calls authenticated by the VM's service account; dashed grey = images loaded straight from the server.

Component	Where it runs	Why this choice
React app	Cloudflare Pages (free)	Global CDN, free HTTPS, a memorable name (`miesign.pages.dev`) without buying a domain.
API proxy	Pages Function, 10 lines	Puts the API on the same origin as the app: no CORS, one public address.
Django API	GCE e2-micro, always-free tier	Never sleeps (no cold starts for reviewers); real disk keeps SQLite and photos; signs in to Vertex AI with its own service account — no key files.
HTTPS on the VM	Caddy + Let's Encrypt via sslip.io	Automatic certificates without owning a domain or running a tunnel.
AI	Vertex AI, location `global`	Gemini text, search-grounding and image editing under one Google Cloud project and its free credits.

Backend layers

The backend follows a strict layering. Each layer may only call the layer below it, which keeps business rules in one place and makes every module testable on its own.

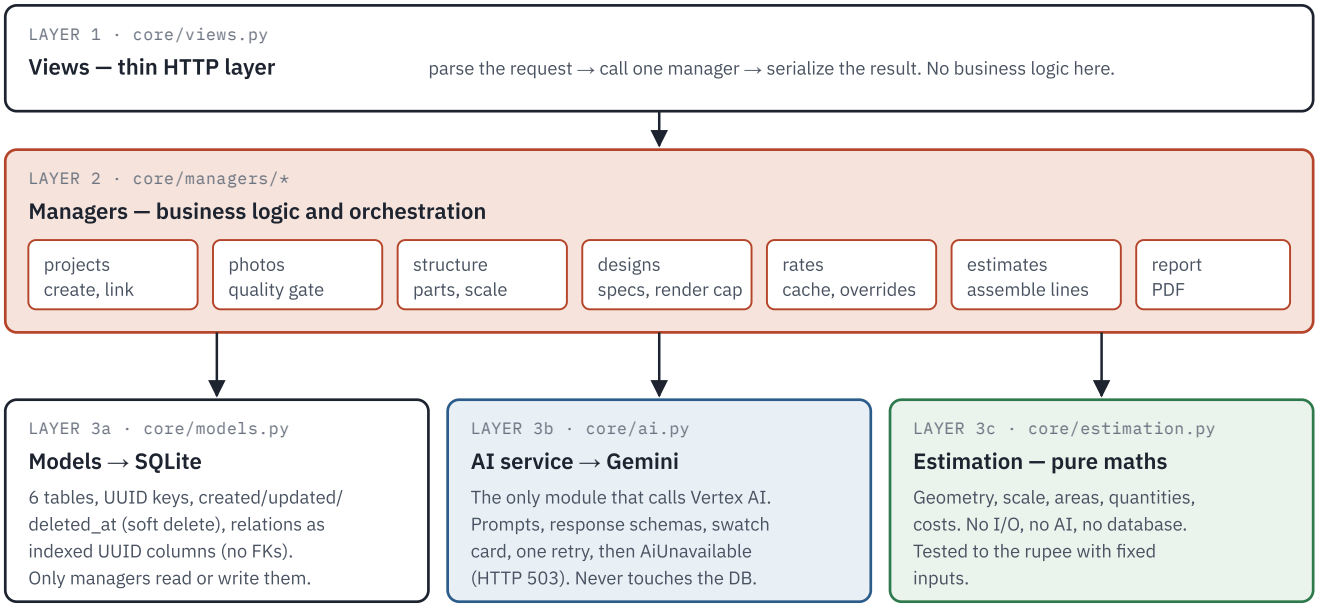


Fig 3.1 Backend layering. Arrows show the only allowed call directions.

Error contract

Every failure is an `AppError(code, message, status)` and renders as `{"error": {"code", "message", "status", "details"}}`. The frontend shows `message` directly, so every message is written for a homeowner. Codes include `INVALID_IMAGE`, `PHOTO_NOT_USABLE`, `EMPTY_DESIGN`, `RENDER_LIMIT` (429) and `AI_UNAVAILABLE` (503).

Configuration

Every tunable is an environment variable with a safe default (`backend/.env.example`): models, GCP project, render cap (8), rate cache windows (30 days web, 24 h estimated), minimum photo side (400 px), blur threshold, upload size (15 MB), photos per project (5).

Data model

Six tables. A **Project** is one house and the shareable unit — its UUID is the link. Everything else hangs off `project_id`, except the city-level rate cache, which is shared by all projects in a city.

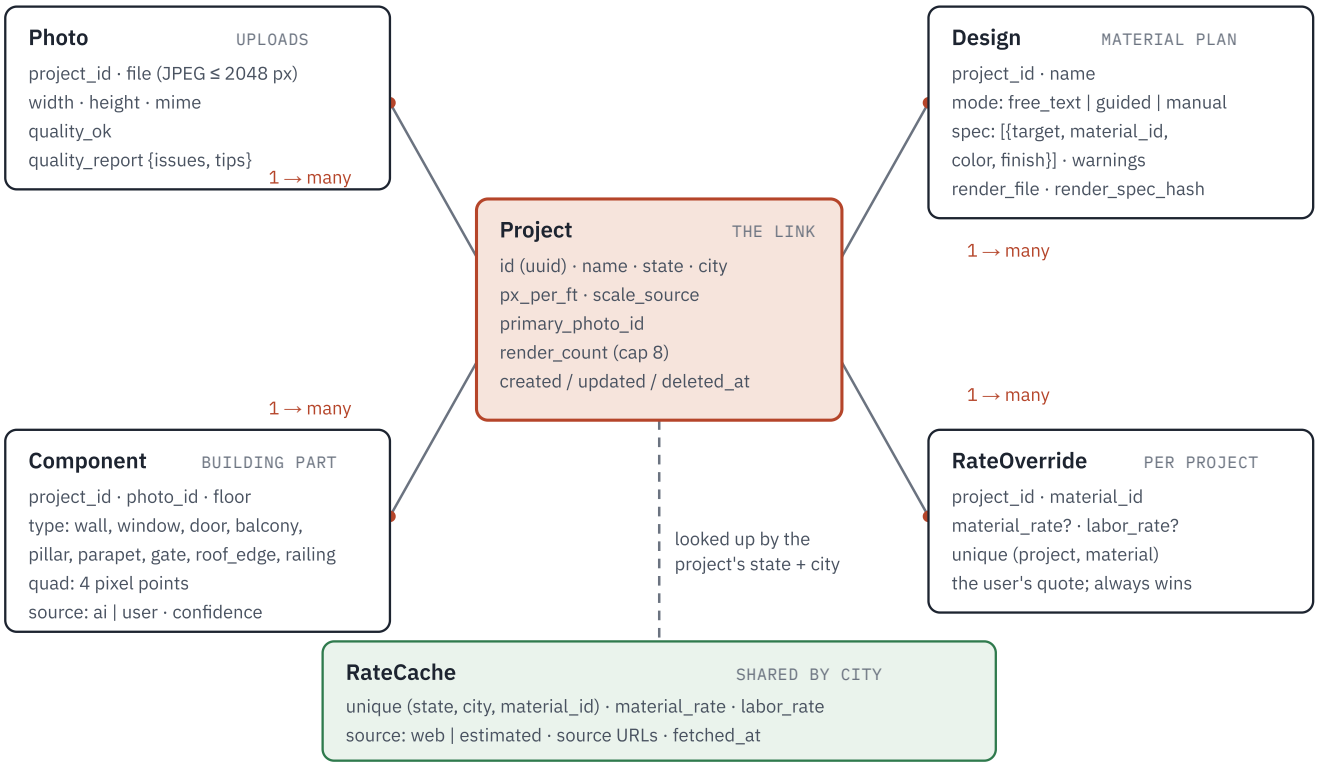


Fig 4.1 Entities and relations. Relations are indexed UUID columns enforced in the managers (no database foreign keys); every table has `id`, `created_at`, `updated_at`, `deleted_at`.

Why these choices

- **Design spec as JSON** — a design is a list of `{target, material_id, color, finish}` assignments where target is a part type (`type:wall`) or one part. The same shape comes from free text, guided answers, the enhancer flow and manual picks, so rendering and estimation have one input format.
- **render_spec_hash** — SHA-256 of the cleaned spec. An unchanged design reuses its saved render and does not spend a preview.
- **Soft delete everywhere** — re-detecting parts or editing them never destroys history, and a half-finished operation can be rolled back atomically.
- **RateCache shared by city** — a web search for Ludhiana paint rates benefits every Ludhiana project for 30 days; failed searches fall back to tier estimates cached for only 24 hours so the search is retried soon.

API surface (19 endpoints)

RPC style: reads are `GET`, every mutation is `POST`; ids travel as query parameters; bodies and responses are snake_case JSON.

Method	Path	Body	Returns
POST	`/api/project/create`	{name?}	project
GET	`/api/project?id=`		project (photos, parts, designs, render counts)
POST	`/api/project/update?id=`	{name?, state?, city?}	project
POST	`/api/photo/upload?project_id=`	multipart file	project
POST	`/api/photo/set-primary?project_id=`	{photo_id}	project
POST	`/api/structure/detect?project_id=`		project with detected parts
POST	`/api/structure/save?project_id=`	{components: [...]}	project
POST	`/api/scale/set?project_id=`	{component_id, real_height_ft} | {use_default}	project
GET	`/api/catalog`		{materials: [12]}
GET	`/api/locations`		{states: [{name, cities: [{name, tier}]}]} (91 cities)
GET	`/api/rates?state=&city=&project_id=`		{rates: [row]}
POST	`/api/rates/override?project_id=`	{material_id, material_rate?, labor_rate?}	{rates: [row]}
POST	`/api/design/from-text?project_id=`	{text} 3–1000 chars	{designs: [design]}
POST	`/api/design/enhance?project_id=`	{text}	{text} rewritten brief (nothing saved)
POST	`/api/design/recommend?project_id=`	budget, climate, style (14), maintenance, color_mood?, keep_unchanged?	{designs: [up to 3]}
POST	`/api/design/save?project_id=`	{id?, name, spec}	{designs: [design]}
POST	`/api/design/render?id=`		design with render_url
GET	`/api/estimate?design_id=`		scale, lines, subtotals, grand_total, assumptions
GET	`/api/report?design_id=`		application/pdf

AI integration and guardrails

Six Gemini tasks, all in `core/ai.py`. Each one is told exactly what it may return, and the server checks the result before anything is saved or priced.

Task	Model	What it returns	Guardrails
Photo check	gemini-3.5-flash	is_house_exterior, facade_visible, issues, tips	Runs only after cheap local checks (≥ 400 px short side, blur variance) pass; only blocking problems reject a photo.
Part detection	gemini-3.5-flash	boxes [ymin, xmin, ymax, xmax] on a 0–1000 grid with a part type	Response schema; boxes clamped to the image; a fallback wall if none found; walls > 80 % inside a bigger wall dropped.
Design from words / suggestions	gemini-3.5-flash	design spec(s)	Schema enums limit materials to the catalog and targets to parts present; named shades map to Miesign's 52 hex codes; server re-validates and lists anything dropped.
Idea enhancer	gemini-3.5-flash	a rewritten brief (text)	Only catalog material names and Miesign shade names; keeps everything the user asked for; never restyles windows or doors; nothing saved.
City rates	gemini-3.5-flash + Google Search grounding	material and labour rate per material, with source URLs	Accepted only within 0.3×–3× of the catalog base rate; otherwise a tier estimate; cached 30 days (24 h for estimates).
Photo redesign	gemini-3.1-flash-image	the edited photo (PNG)	JSON prompt with hard rules (same camera, no added or moved windows/doors/pillars), each change located by its detected boxes, a swatch-card image of exact colours; 8 per project, refunded on failure, cached by spec hash.

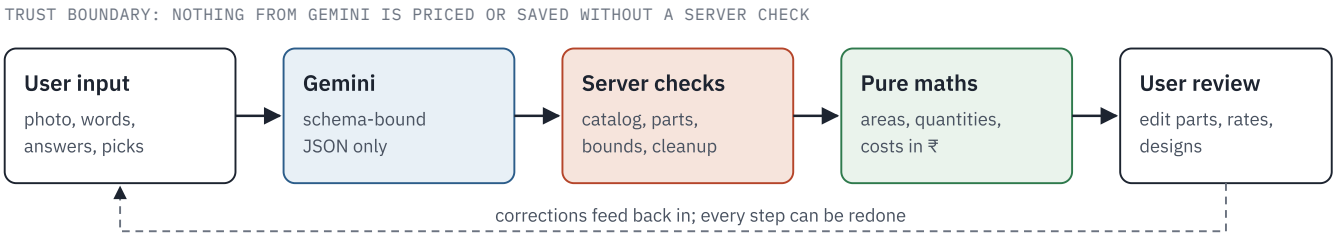


Fig 6.1 How AI output flows through the system.

Request lifecycle: rendering a design

The most involved request. It shows the render cap, the cache, the refund and how the prompt is built.

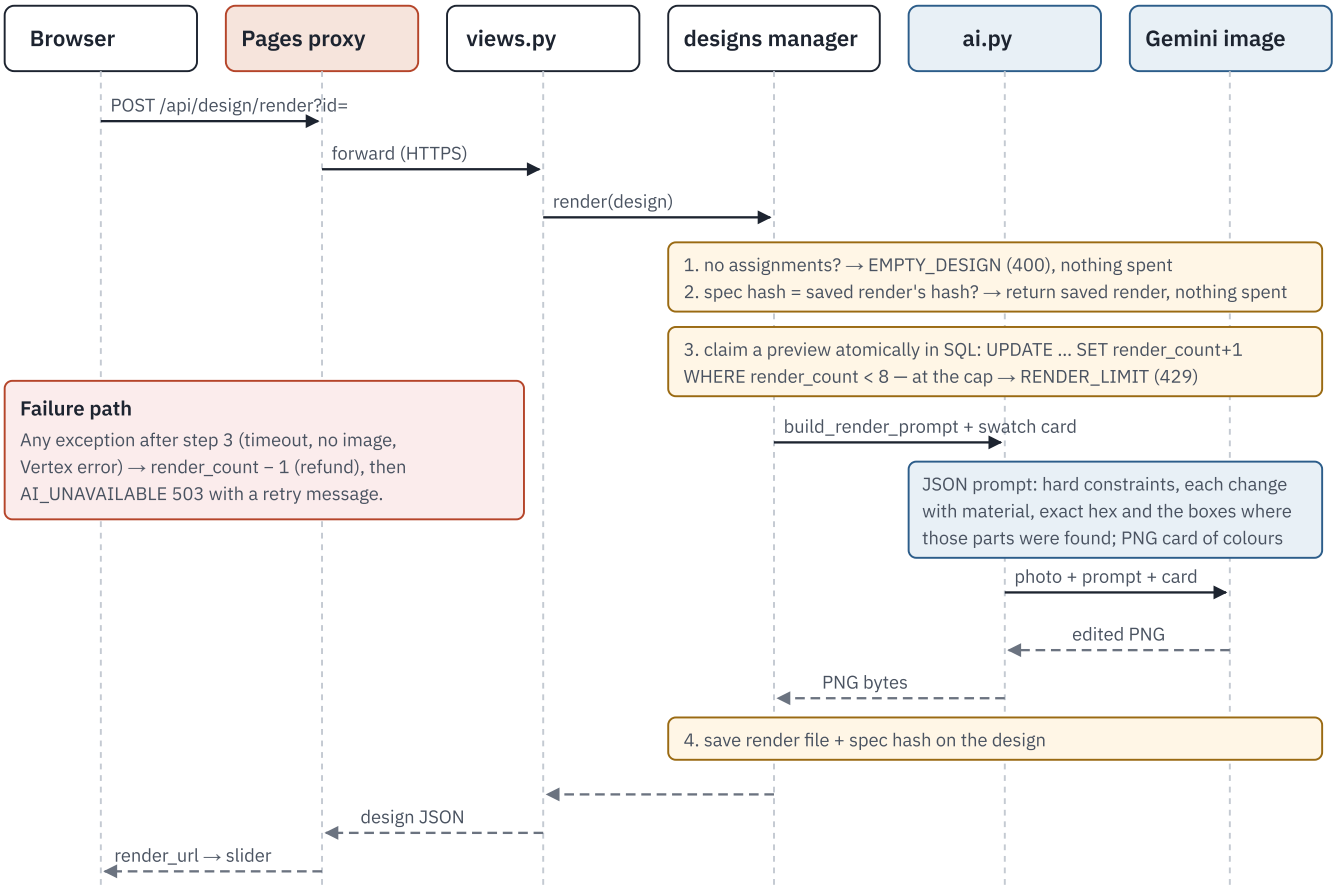


Fig 7.1 Sequence for `POST /api/design/render`. Solid = request, dashed = response. Typical duration 10–40 s; the browser shows a fluid loader in the design's colours meanwhile.

8 · FRONTEND

Frontend architecture

A React 18 + TypeScript (strict) single-page app built with Vite. There is no global state library: `App.tsx` owns the project, catalog and current step; each step is one component that receives the project and returns an updated one.

- **HomePage** — explains the product with a real before/after; a `?p=` link skips it.
- **Six step components** — Photo, Parts, Design, Preview, Estimate, Report; a step unlocks when the data it needs exists (`lib/steps.ts`).
- **Shared components** — StepRail, RegionCanvas (Konva part editor), ShadePicker (52 shades, 8 families, any colour), MagicLoader, CompareSlider, RateCell, CitySelect.
- **lib/api.ts** — the only place that converts camelCase ↔ snake_case and turns the error shape into readable messages.
- **Project in the URL** — creating a project writes `?p=<uuid>` to the address bar at once; reloading or sharing that link resumes at the furthest unlocked step.

Design system

Tokens only (`tokens.css`): cool paper background, ink text, one brick accent, Newsreader headings, IBM Plex Sans body, IBM Plex Mono numbers. Layout tested at 320, 375, 768 and 1440 px with no sideways scrolling and 44 px touch targets on phones. Animations respect `prefers-reduced-motion`.

Waiting states

Every AI call shows elapsed seconds. Photo check, part detection and rendering show the MagicLoader — fluid blobs in the design's own colours, a light sweep and sparkles over the image — and the enhancer gives the text box a moving colour border.

9 · OPERATIONS

Concurrency, cost and security

Concern	How it is handled
Several users at once	gunicorn 2 workers × 4 threads: 8 requests can wait on Gemini in parallel. SQLite waits up to 20 s for a write lock instead of failing. Tested with 8 simultaneous project creations, all succeeded. Projects are independent, so users never see each other's data.
Double clicks and races	The frontend caches the in-flight "create project" promise; the render cap is claimed with one atomic SQL update; re-detection replaces parts in one transaction.
AI cost control	8 renders per project, refunded on failure, cached by spec hash; rates cached per city for 30 days; cheap local photo checks run before any Gemini call. A ₹850 budget alert emails at 50 %, 90 % and 100 %.
Credentials	None stored. The VM's service account (role `aiplatform.user`) signs every Vertex request; the Django secret key is generated on the server; `.env` files are git-ignored.
Untrusted input	Uploads re-encoded to JPEG (EXIF rotation applied, ≤ 2048 px, ≤ 15 MB); part quads clamped to the image; every JSON body validated; PDF text escaped.
Access	No accounts, so a non-technical user can start without signing up: the project link is the credential. Documented as a limitation.

Technology stack

Layer	Technology
Frontend	React 18, TypeScript (strict), Vite 5, Konva / react-konva, native fetch, Vitest
Backend	Python 3, Django 5.2, Django REST Framework, Pillow, ReportLab, google-genai, pytest
AI	Gemini 3.5 Flash (text, vision, search grounding), Gemini 3.1 Flash Image (editing) on Vertex AI
Data	SQLite, local media disk
Hosting	Cloudflare Pages + Pages Functions; Google Compute Engine e2-micro (Debian 12), Caddy, gunicorn, systemd
Quality	108 backend tests, 22 frontend tests, live AI checks on real photos, browser walkthroughs at 4 widths